5 Count to 10

5.0.1 While loops

Presenting our first *control structure*. Ordinarily the computer starts with the first line and then goes down from there. Control structures change the order that statements are executed or decide if a certain statement will be run. Here's the source for a program that uses the while control structure:

```
a = 0            # FIRST, set the initial value of the variable a to 0(zero).
while a < 10:    # While the value of the variable a is less than 10 do the
 following:
    a = a + 1    # Increase the value of the variable a by 1, as in: a = a + 1!
    print(a)     # Print to screen what the present value of the variable a is.
                 # REPEAT! until the value of the variable a is equal to 9!? See
 note.

                 # NOTE:
                 # The value of the variable a will increase by 1
                 # with each repeat, or loop of the 'while statement BLOCK'.
                 # e.g. a = 1 then a = 2 then a = 3 etc. until a = 9 then...
                 # the code will finish adding 1 to a (now a = 10), printing the

                 # result, and then exiting the 'while statement BLOCK'.
                 #              --
                 # While a < 10: |
                 #     a = a + 1 |<--[ The while statement BLOCK ]
                 #     print (a) |
                 #               --
```

And here is the extremely exciting output:

```
1
2
3
4
5
6
7
8
9
10
```

(And you thought it couldn't get any worse after turning your computer into a five-dollar calculator?)

So what does the program do? First it sees the line `a = 0` and sets `a` to zero. Then it sees `while a < 10:` and so the computer checks to see if `a < 10`. The first time the computer sees this statement, `a` is zero, so it is less than 10. In other words, as long as `a` is less than ten, the computer will run the tabbed in statements. This eventually makes `a` equal to

ten (by adding one to `a` again and again) and the `while a < 10` is not true any longer. Reaching that point, the program will stop running the indented lines.

Always remember to put a colon "**:**" at the end of the `while` statement line!

Here is another example of the use of `while`:

```
a = 1
s = 0
print('Enter Numbers to add to the sum.')
print('Enter 0 to quit.')
while a != 0:
    print('Current Sum:', s)
    a = float(input('Number? '))
    s = s + a
print('Total Sum =', s)
```

```
Enter Numbers to add to the sum.
Enter 0 to quit.
Current Sum: 0
Number? 200
Current Sum: 200.0
Number? -15.25
Current Sum: 184.75
Number? -151.85
Current Sum: 32.9
Number? 10.00
Current Sum: 42.9
Number? 0
Total Sum = 42.9
```

Notice how `print('Total Sum =', s)` is only run at the end. The `while` statement only affects the lines that are indented with whitespace. The `!=` means does not equal so `while a != 0:` means as long as `a` is not zero run the tabbed statements that follow.

Note that `a` is a floating point number, and not all floating point numbers can be accurately represented, so using `!=` on them can sometimes not work. Try typing in 1.1 in interactive mode.

Infinite loops or Never Ending Loop

Now that we have while loops, it is possible to have programs that run forever. An easy way to do this is to write a program like this:

```
while 1 == 1:
   print("Help, I'm stuck in a loop.")
```

The "`==`" operator is used to test equality of the expressions on the two sides of the operator, just as "`<`" was used for "less than" before (you will get a complete list of all comparison operators in the next chapter).

This program will output `Help, I'm stuck in a loop.` until the heat death of the universe or you stop it, because 1 will forever be equal to 1. The way to stop it is to hit the Control (or *Ctrl*) button and *C* (the letter) at the same time. This will kill the program. (Note: sometimes you will have to hit enter after the Control-C.) On some systems, nothing will stop it, short of killing the process–so avoid!

5.0.2 Examples

Fibonacci sequence

Fibonacci-method1.py

```
# This program calculates the Fibonacci sequence
a = 0
b = 1
count = 0
max_count = 20

while count < max_count:
    count = count + 1
    print(a, end=" ")  # Notice the magic end=" " in the print function
 arguments
                       # that keeps it from creating a new line.
    old_a = a    # we need to keep track of a since we change it.
    a = b
    b = old_a + b
print()  # gets a new (empty) line.
```

Output:

```
 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181
```

Note that the output is on a single line because of the extra argument `end=" "` in the `print` arguments.

Fibonacci-method2.py

```
# Simplified and faster method to calculate the Fibonacci sequence
a = 0
b = 1
count = 0
max_count = 10

while count < max_count:
    count = count + 1
    print(a, b, end=" ")  # Notice the magic end=" "
    a = a + b
    b = a + b
print()  # gets a new (empty) line.
```

Output:

```
 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181
```

Fibonacci-method3.py

```
a = 0
b = 1
count = 0
maxcount = 20

#once loop is started we stay in it
while count < maxcount:
    count += 1
    olda = a
    a = a + b
```

```
    b = olda
    print(olda,end=" ")
print()
```

Output:

```
 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181
```

Enter password

Password.py

```
# Waits until a password has been entered.  Use Control-C to break out without
# the password

#Note that this must not be the password so that the
# while loop runs at least once.
password = str()

# note that != means not equal
while password != "unicorn":
    password = input("Password: ")
print("Welcome in")
```

Sample run:

```
Password: auo
Password: y22
Password: password
Password: open sesame
Password: unicorn
Welcome in
```

5.0.3 Exercises

Write a program that asks the user for a Login Name and password. Then when they type ”lock”, they need to type in their name and password to unlock the program.

Solution

Write a program that asks the user for a Login Name and password. Then when they type ”lock”, they need to type in their name and password to unlock the program.

```
name = input("What is your UserName: ")
password = input("What is your Password: ")
print("To lock your computer type lock.")
command = None
input1 = None
input2 = None
while command != "lock":
    command = input("What is your command: ")
while input1 != name:
```